Next.js

Setup & Onboarding Guide

A practical guide for the team — from installation to your first running app

📋 **What You Will Learn**

- How to install Node.js and verify your environment
- How to scaffold a new Next.js project with create-next-app
- The folder structure and what each file does
- How to run the development server and navigate the demo app
- Key concepts: App Router, Server vs Client Components, Dynamic Routes

Version: Next.js 14 (App Router) | April 2025

1. Prerequisites

Before creating a Next.js project, make sure the following are installed on your machine.

1.1 Node.js

Next.js requires Node.js version 18.17 or later.

Check if Node.js is installed:

```
node --version
# Expected output: v18.17.0 or higher
```

If Node.js is not installed, download it from:

- https://nodejs.org — choose the LTS version
- Or use a version manager: nvm (Mac/Linux) or nvm-windows

> 💡 **Tip: Using nvm**
> nvm install 20 # install Node.js 20
> nvm use 20 # activate it
> node --version # confirm

1.2 npm / pnpm / yarn

npm comes bundled with Node.js. Verify it works:

```
npm --version
# Expected: 9.x or 10.x
```

This guide uses npm. You can substitute pnpm or yarn if your team prefers.

Tool	Install Command	Speed	Notes
npm	(bundled with Node)	Fast	Default choice
pnpm	npm install -g pnpm	Very Fast	Saves disk space
yarn	npm install -g yarn	Fast	Classic or Berry

1.3 Code Editor

VS Code is recommended. Install the following extensions for the best experience:

- ESLint — real-time linting
- Prettier — auto-formatting
- Tailwind CSS IntelliSense — if using Tailwind
- TypeScript — built in, but ensure it is enabled

2. Creating a New Next.js Project

The fastest way to bootstrap a project is the official create-next-app CLI.

2.1 Running the CLI

```
npx create-next-app@latest my-demo-app

# You will be prompted to choose options:
# TypeScript?           → Yes
# ESLint?               → Yes
# Tailwind CSS?         → No (for this demo)
# src/ directory?       → No
# App Router?           → Yes  ← IMPORTANT
# Customize import alias? → No (press Enter)
```

> ⚠️ **Always choose App Router**
> When the CLI asks 'Would you like to use App Router?', always answer Yes.
> The App Router (app/) is the modern, recommended way to build Next.js apps.
> The older Pages Router (pages/) is in maintenance mode.

2.2 Opening the Project

```
cd my-demo-app
code .          # open in VS Code
```

2.3 Installing Dependencies & Running Dev Server

```
npm install     # install packages (only needed first time)
npm run dev     # start development server

# Open your browser: http://localhost:3000
```

You should see the default Next.js welcome page. The dev server has hot reload — changes to files appear instantly without refreshing the browser.

3. Project Structure

Here is what the scaffolded project looks like and what each part does:

```
my-demo-app/
├── app/                    ← All routes live here
│   ├── layout.tsx          ← Root layout (wraps every page)
│   ├── page.tsx            ← Home page → route: /
│   ├── about/
│   │   └── page.tsx        ← About page → route: /about
│   ├── blog/
│   │   ├── page.tsx        ← Blog index → route: /blog
│   │   └── [slug]/
│   │       └── page.tsx ← Dynamic route → /blog/:slug
│   └── dashboard/
│       └── page.tsx        ← Dashboard → route: /dashboard
├── components/
│   └── Counter.tsx         ← Reusable Client Component
├── lib/
│   └── data.ts             ← Shared data-fetching utilities
├── public/                 ← Static assets (images, fonts, etc.)
├── next.config.js          ← Next.js configuration
├── package.json            ← Dependencies and scripts
└── tsconfig.json           ← TypeScript configuration
```

3.1 The Routing Rules

Next.js uses the file system as the router. The rules are simple:

File Path	URL Route	Notes
app/page.tsx	/	Home page
app/about/page.tsx	/about	Static page
app/blog/page.tsx	/blog	Blog index
app/blog/[slug]/page.tsx	/blog/:slug	Dynamic — any slug
app/shop/[cat]/[id]/page.tsx	/shop/:cat/:id	Multiple dynamic segments

3.2 Special Files

Certain filenames have special meaning in Next.js App Router:

Filename	Purpose
page.tsx	Creates a publicly accessible route
layout.tsx	Wraps child routes — persists across navigation
loading.tsx	Shows a loading UI while the page renders

error.tsx	Error boundary for the route segment
not-found.tsx	Custom 404 page for the segment
route.ts	API endpoint (like an Express handler)

4. Key Concepts

4.1 Server Components vs Client Components

This is the most important concept in Next.js App Router.

Feature	Server Component	Client Component
Default?	Yes	Add 'use client' directive
Runs in	Server (Node.js)	Browser (+ server for SSR)
useState / useEffect	No	Yes
Event Handlers (onClick)	No	Yes
Access DB / File System	Yes	No
Bundle size impact	None	Adds JS to the browser
Best for	Data fetching, static UI	Interactivity, forms, hooks

✅ **Best Practice**
Default to Server Components. Only add 'use client' when you need interactivity (hooks, event handlers, browser APIs).
Push client components to the leaves of the component tree.

4.2 Data Fetching

In Server Components, you fetch data directly in the component — no useEffect, no useState:

```
// app/page.tsx — Server Component
export default async function HomePage() {
  // This runs on the SERVER — can access DB, env vars, etc.
  const data = await fetch('https://api.example.com/stats');
  const stats = await data.json();

  return <div>{stats.users} users</div>;
}
```

4.3 Dynamic Routes

Create a folder with square brackets to make a dynamic segment:

```
// app/blog/[slug]/page.tsx
export default function PostPage({ params }: { params: { slug: string } }) {
  // params.slug = the value from the URL
  // e.g. URL /blog/hello-world → params.slug = 'hello-world'
  return <h1>Post: {params.slug}</h1>;
}
```

4.4 API Routes

Next.js lets you build REST API endpoints inside the same project — no separate Express server needed. Create a route.ts file inside any app/ folder:

```
// app/api/blog/route.ts  →  accessible at: /api/blog
import { NextRequest, NextResponse } from 'next/server';

// Named export per HTTP method: GET, POST, PUT, DELETE, PATCH
export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url);
  const id = searchParams.get('id'); // e.g. /api/blog?id=5

  const res = await fetch(`https://jsonplaceholder.typicode.com/posts/${id}`);
  const data = await res.json();

  return NextResponse.json({ success: true, data });
}

export async function POST(request: NextRequest) {
  const body = await request.json();
  // validate, write to DB, etc.
  return NextResponse.json({ created: body }, { status: 201 });
}
```

Feature	Details
File name	Must be route.ts (not page.tsx)
HTTP methods	Export GET, POST, PUT, DELETE, PATCH as named functions
Request access	Use NextRequest — read URL, headers, body, cookies
Response	Return NextResponse.json() with optional status code
Caching	Pass { next: { revalidate: 3600 } } to cache fetch calls
Cannot coexist	A folder cannot have both page.tsx AND route.ts

🌐 **Free API Used in This Demo: JSONPlaceholder**

URL: https://jsonplaceholder.typicode.com
Auth: None — completely free, no API key required
Data: 100 fake blog posts, 10 users, 200 comments
Purpose: The /api/blog route fetches from JSONPlaceholder and reshapes the data before returning it to the frontend.
Note: POST requests are simulated — data is not actually stored.

API Endpoints in the Demo

Method	URL	Description
GET	/api/blog	All posts, paginated (default: 6 per page)
GET	/api/blog?page=2&limit=5	Page 2, 5 posts per page
GET	/api/blog?id=1	Single post by ID
GET	/api/blog?userId=3	Filter posts by author (userId 1–10)
POST	/api/blog	Create post — body: { title, content }

Example: test the POST endpoint from your terminal:

```
curl -X POST http://localhost:3000/api/blog \
  -H 'Content-Type: application/json' \
  -d '{"title": "My Post", "content": "Hello world"}'

# Response:
# { "success": true, "data": { "id": 101, "title": "My Post", ... } }
```

Sample GET /api/blog Response

```
{
  "success": true,
  "pagination": {
    "page": 1, "limit": 6, "total": 100,
    "totalPages": 17, "hasNext": true, "hasPrev": false
  },
  "data": [
    {
      "id": 1,
      "title": "Sunt aut facere repellat provident",
      "excerpt": "Quia et suscipit suscipit recusandae…",
      "author": "Author 1",
      "slug": "1-sunt-aut-facere-repellat-provid",
      "readTimeMinutes": 1
    }
  ],
  "source": "https://jsonplaceholder.typicode.com"
}
```

5. Running the Demo Project

The attached demo project is pre-built to showcase all the concepts above. Follow these steps to run it:

Step 1 — Clone or Copy the Project

```
# If shared as a ZIP, extract it and open the folder
cd nextjs-demo
```

Step 2 — Install Dependencies

```
npm install
```

Step 3 — Start the Dev Server

```
npm run dev
# App runs at: http://localhost:3000
```

Step 4 — Explore the Routes

URL	What to Look For
localhost:3000	Home — server-fetched stats, zero client JS
localhost:3000/about	Static server component, per-page metadata
localhost:3000/blog	Live data from /api/blog, server-side pagination
localhost:3000/blog/1-sunt-aut	Dynamic route — API call + params debug box
localhost:3000/dashboard	Server page composing a Client Component (Counter)
localhost:3000/api/blog	Raw JSON response — open in browser to inspect
localhost:3000/api/blog?id=5	Single post JSON — notice reshaped fields
localhost:3000/api/blog?page=2	Pagination in action — check pagination object

🔍 **Developer Tools Tip**
Open Chrome DevTools → Network tab → filter by JS.
Notice that the Home and About pages have almost no JS.
Only the Dashboard page (which has the Client Component) loads extra JS.
This shows the benefit of the Server Component model.

6. Common Commands

Command	What It Does
npm run dev	Start dev server with hot reload at localhost:3000
npm run build	Build production bundle — catches TypeScript errors
npm run start	Serve the production build locally
npm run lint	Run ESLint on all files
npx next info	Print environment info (useful for bug reports)

7. Troubleshooting

Problem	Solution
Port 3000 already in use	Run: npm run dev -- --port 3001
'next' is not recognized	Run: npm install (dependencies not installed)
TypeScript errors on build	Check tsconfig.json — strict mode catches many issues
Page not found (404)	Ensure the folder has a page.tsx file inside it
Client Component error in Server Component	Add 'use client' at top of the file
/api/blog returns fetch error	Ensure NEXT_PUBLIC_BASE_URL=http://localhost:3000 in .env.local
API returns 502 Bad Gateway	JSONPlaceholder may be temporarily down — retry in a minute

8. Next Steps

Once comfortable with the demo, explore these areas:

- Add Tailwind CSS: npx create-next-app@latest with Tailwind selected
- Connect a database: Use Prisma or Drizzle ORM with a Postgres/SQLite DB
- Add authentication: Try NextAuth.js or Clerk
- Deploy to production: Push to GitHub and connect to Vercel (free tier)
- Extend the blog API: Add PUT and DELETE handlers to app/api/blog/route.ts
- Add middleware: Create middleware.ts to protect /dashboard behind auth
- Connect a real database: Use Prisma or Drizzle ORM with Postgres/SQLite

📚 **Official Resources**

- Documentation: https://nextjs.org/docs
- Learn Course: https://nextjs.org/learn
- Examples: https://github.com/vercel/next.js/tree/canary/examples
- Playground: https://stackblitz.com/fork/nextjs